

Interfaces und abstrakte Klassen – Teil 2

`instanceof`, `interface` und `super`

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 18. Dezember 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Interfaces und abstrakte Klassen

 Plan für diese Woche

 Abstrakte und konkrete Klassen

 Abstrakte Methoden

 Pair Programming

 Die Klasse Object

 Typumwandlung

 Referenzvariablen

 Mehrfachvererbung

 Interfaces

 Pair Programming

 Superklassenversion

 Spaß

📖 Interfaces und abstrakte Klassen

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

↩️ Wenn Sie eine Objektreferenz vom Typ Superhero haben, die auf eine Instanz von Batman zeigt, wie können Sie diese Referenz in den tatsächlichen Typ umwandeln?

🧙 Warum ist der Typ einer Referenzvariablen so wichtig?

😡 Warum gibt es in Java keine Mehrfachvererbung?

👤 Was haben Interfaces mit abstrakten Klassen und Methoden gemeinsam?

UP! Wie rufen Sie die Superklassenversion einer Methode auf?

🙋 Welche Fragen haben Sie zu diesem Kapitel?

8 Interfaces und abstrakte Klassen

Ernsthafte Polymorphie



Vererbung ist nur der Anfang. Um Polymorphie nutzen zu können, benötigen wir Interfaces (und damit meinen wir keine GUIs). Wir müssen über einfache Vererbung hinausgehen, um eine Stufe der Flexibilität und Erweiterbarkeit zu erreichen, die Sie nur erhalten, wenn man Schnittstellendefinitionen als Ausgangsbasis für den Entwurf und die Programmierung nimmt. Einige der coolsten Teile von Java wären ohne Schnittstellen überhaupt nicht möglich. Auch wenn Sie die Schnittstellen nicht selbst entwickeln müssen, werden Sie um ihre Nutzung nicht herumkommen. Und das werden Sie *wollen*. Sie werden mit Ihnen entwerfen *müssen*. ***Sie werden sich fragen, wie Sie bisher überhaupt ohne Schnittstellen leben konnten.*** Aber was ist das eigentlich? Eine Schnittstelle ist eine zu 100 Prozent abstrakte Klasse, also eine Klasse, die nicht instanziiert werden kann. Und wozu soll das gut sein? Das werden Sie in wenigen Augenblicken sehen. Wenn Sie an das Ende des vorherigen Kapitels zurückdenken und wie wir polymorphe Argumente genutzt haben, damit eine einzelne Vet-(Tierarzt-)Methode verschiedene Animal-Unterklassen übernehmen konnte, dann war das gerade mal der Anfang. Schnittstellen sind das ***Poly*** in Polymorphie, das ***ab*** in abstrakt, das ***Koffein*** in Java.

Typumwandlung

„Was ist, wenn ich eine Referenz vom Typ Superhero habe und ich weiß, dass das Objekt dahinter vom Typ Batman ist – wie kann ich `Batman.callRobin()` aufrufen?“

```
Superhero[] team = Director.assembleTeam();  
Superhero batman = team[1];  
batman.callRobin(); // compilation error 💣
```

Robin – neugierige Student:in im 1. Semester



```
1 import java.util.*;
2
3 public class CastDemo {
4     public static void main(String[] args) {
5         List list = List.of(1, "two");
6
7         Object one = list.get(0);
8         System.out.println("Element at index 0: " + one);
9
10        Object two = list.get(1);
11        System.out.println("Element at index 1: " + two);
12
13        // #1 We want to print the word 'two' as 'TWO'.
14        // System.out.println(two.toUpperCase()); // error: compilation failed (cannot find symbol)
15        String twoAsString = (String) two;
16        System.out.println("'two' in upper case: " + twoAsString.toUpperCase());
17
18        // #2 We can't "convert" (that is, cast) Integer to String, can we?
19        // String oneAsString = (String) one; // ClassCastException
20
21        // #3 We can explicitly test if element is an instance of String:
22        for (Object element : list) {
23            System.out.println("Element: " + element);
24            if (element instanceof String) {
25                String elementAsString = (String) element;
26                System.out.println("Element in upper case: " + elementAsString.toUpperCase());
27            }
28        }
29    }
30 }
31
```

STDIN

Input for the program (Optional)

Output:

```
Element at index 0: 1
Element at index 1: two
'two' in upper case: TWO
Element: 1
Element: two
Element in upper case: TWO
```




„Doch Vorsicht! Die Verwendung dieses Operators wie Typumwandlung im Allgemeinen ist eher die Ausnahme als die Regel. Verwenden Sie stattdessen Polymorphie!“

„Aha! Wenn ich mir nicht sicher bin, kann ich mit dem Operator `instanceof` überprüfen, ob ein Objekt eine Instanz einer bestimmten Klasse ist.“

Simone – aufgeweckte Student:in im 1. Semester



Polymorphie statt Downcast

Typumwandlung

```
1 public abstract class Base { /* some abstract base class */ }
2
3 public class ImplA extends Base { /* some concrete subclass */ }
4
5 public class ImplB extends Base { /* another concrete subclass */ }
6
7 public class Main {
8
9     public static void main (String[] args) {
10         Base base = new ImplA();
11         if (base instanceof ImplA) {
12             ImplA implA = (ImplA) base;
13             // ImplA-specific code
14         } else if (base instanceof ImplB) {
15             ImplB implB = (ImplB) base;
16             // ImplB-specific code
17         }
18         // What about future ImplC-specific code?
19     }
```



```
1 public abstract class Base {
2     public abstract void doit();
3 }
4
5 public class ImplA extends Base {
6     public void doit() {
7         // ImplA-specific code
8     }
9 }
10
11 public class ImplB extends Base {
12     public void doit() {
13         // ImplB-specific code
14     }
15 }
16
17 public class ImplC extends Base {
18     public void doit() {
19         // future ImplC-specific code
20     }
21 }
```



! Quiz

↪ Typumwandlung

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Mit Hilfe eines Casts können Sie ein Objekt explizit von einer Oberklasse in eine Unterklasse umwandeln. Daher der Name Downcast.

Richtig!

Ein Downcast ermöglicht den Zugriff auf die spezifischen Methoden und Eigenschaften der Unterklasse.

Richtig!

Ein solcher Cast ist immer sicher möglich,
da Java eine stark typisierte Sprache ist.

Wenn das Objekt nicht wirklich eine Instanz der Zielunterklasse ist, tritt ein Fehler auf.

Vor dem Cast sollte der `instanceof`-Operator verwendet werden, um die Typkompatibilität zu prüfen.

Richtig!

Vor der Anwendung eines Casts sollten andere typsichernde Techniken wie Polymorphie in Betracht gezogen werden.

Richtig!

 Welcher **eine** Gedanke zu
„ Typumwandlung“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren



Referenzvariablen

Typsicherheit

🥚 Referenzvariablen

- Sie wissen nun, wie wichtig in Java die Methoden in der Klasse der **Referenzvariablen** sind.
- Sie können nur dann eine Methode auf einem Objekt aufrufen, wenn die Klasse der **Referenzvariablen** diese Methode enthält.
- Stellen Sie sich die öffentlichen Methoden in Ihrer Klasse als Ihren Vertrag vor, Ihr Versprechen an die Außenwelt, was diese tun können.



Die Programmiersprache Java ist eine **statisch typisierte Sprache**, was bedeutet, dass jede Variable und jeder Ausdruck einen Typ hat, der zur Kompilierungszeit bekannt ist.

Java ist auch eine **stark typisierte Sprache**, weil Typen die Werte begrenzen, die eine Variable enthalten oder ein Ausdruck erzeugen kann, die für diese Werte unterstützten Operationen begrenzen und die Bedeutung der Operationen bestimmen.

Starke statische Typisierung hilft bei der **Erkennung von Fehlern zur Kompilierzeit**.

„Der Vertrag kann vom Compiler automatisch und eindeutig überprüft werden. Im Gegensatz zu einer verbalen Zusicherung in irgendeiner Dokumentation.“

Maxi – professionelle Entwickler:in, die andere zu Spitzenleistungen führt



„Und ich denke immer daran, dass der Compiler die Klasse der Referenzvariablen überprüft und nicht die Klasse des Objekts am anderen Ende der Referenz.“

Merle – aufmerksame Student:in im 1. Semester



! ? Quiz

🧸 Referenzvariablen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Unabhängig vom Typ des Objekts selbst können auf einem Objekt nur Methoden aufgerufen werden, die in der Klasse (oder dem Interface) deklariert sind, die (das) als Typ der Referenzvariablen verwendet wird.

Eine Referenzvariable vom Typ `Object` kann also nur zum Aufruf von Methoden verwendet werden, die in der Klasse `Object` definiert sind, unabhängig vom Typ des Objekts, auf das die Referenz verweist.

Richtig!

Wenn eine Methode aufgerufen wird,
verwendet die Methode die Implementierung,
die durch den Referenztyp festgelegt ist.

... die durch den tatsächlichen Objekttyp zur Laufzeit festgelegt ist.

Eine Referenzvariable vom Typ `Object` kann einer Referenzvariablen eines anderen Typs nur mit Hilfe eines Casts (Typumwandlung) zugewiesen werden. Eine Typumwandlung kann verwendet werden, um eine Referenzvariable eines Typs einer Referenzvariablen eines Untertyps zuzuweisen. Die Typumwandlung schlägt jedoch zur Laufzeit fehl, wenn der Typ des Objekts auf dem Heap nicht mit dem Typ der Typumwandlung kompatibel ist.

Beispiel: `Dog d = (Dog) x.getObject(aDog);`

Richtig!

Aus einer `ArrayList<Object>` kommen alle Objekte mit dem Typ `Object` heraus. Sie können also nur mit einer Referenzvariablen vom Typ `Object` referenziert werden, sofern keine Typumwandlung verwendet wird.

Richtig!

 Welcher **eine** Gedanke zu
„ Referenzvariablen“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren



Mehrfachvererbung



Jetzt Seiten 220 bis 225 wiederholen!



Sie haben dafür inklusive Pause 10 Minuten Zeit.

! Quiz

😈 Mehrfachvererbung

🤔 Ich mache einige Aussagen –
sind diese Aussagen richtig
oder falsch?

👍 Bei **richtigen** Aussagen **stehen**
Sie **kurz auf!**

👎 Bei **falschen** Aussagen **bleiben**
Sie **sitzen!**



In Java ist keine Mehrfachvererbung möglich. Dies ist auf den „Deadly Diamond of Death“ zurückzuführen.

Richtig!

Dieses „Diamantproblem“ tritt auf, wenn eine Klasse von mehreren Superklassen erbt, die eine gemeinsame Methode haben. Dies führt zu Mehrdeutigkeiten, da der Compiler nicht weiß, welche Methode der Superklasse ausgeführt werden soll.

Richtig!

Da die Mehrfachvererbung von Klassen nicht erlaubt ist, können Sie nur eine Klasse erweitern. Sie können also nur eine direkte Superklasse verwenden.

Richtig!

In Java gibt es keine Alternativen zur Mehrfachvererbung.

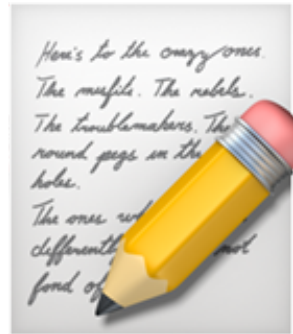
Dafür gibt es Interfaces.



Welcher **eine** Gedanke zu



„Mehrfachvererbung“ war
für Sie am wichtigsten?



Schreiben Sie ihn auf!



Durchatmen und



Sortieren



Interfaces

100%ig abstrakt

Interfaces

- In Java kann mit dem Schlüsselwort **interface** eine Schnittstelle definiert werden. Eine Art Zusicherung, was gemacht werden kann.
- Ein solches Interface deklariert nur abstrakte Methoden.
- Eine konkrete Unterklasse muss also alle Methoden implementieren. Damit ist zur Laufzeit klar, welche konkrete Implementierung aufgerufen wird.
- Auf diese Weise löst Java das Dilemma der Mehrfachvererbung.

<i>Pet</i>
<i>abstract void beFriendly();</i>
<i>abstract void play();</i>

Ein Java-Interface ist wie eine zu einhundert Prozent abstrakte Klasse.

Alle Methoden in einem Interface sind abstrakt, sodass jede Klasse, die EIN Pet IST, die Methoden von Pet implementieren MUSS (etwa durch Überschreiben).

Ein Interface definieren

Interfaces

Verwenden Sie hier das Schlüsselwort
„interface“ anstelle von „class“.

```
public interface Pet {  
    public abstract void beFriendly();  
    public abstract void play();  
}
```

Interface-Methoden
sind implizit öffentlich
und abstrakt.

Alle Methoden eines Interfaces
sind abstrakt. Sie enden daher
mit einem Semikolon und
haben keinen Rumpf (Body).

Ein Interface implementieren

Interfaces

Schreiben Sie hier
„implements“ gefolgt
vom Interface-Namen.

Dog IST EIN Canine und Dog IST EIN Pet.

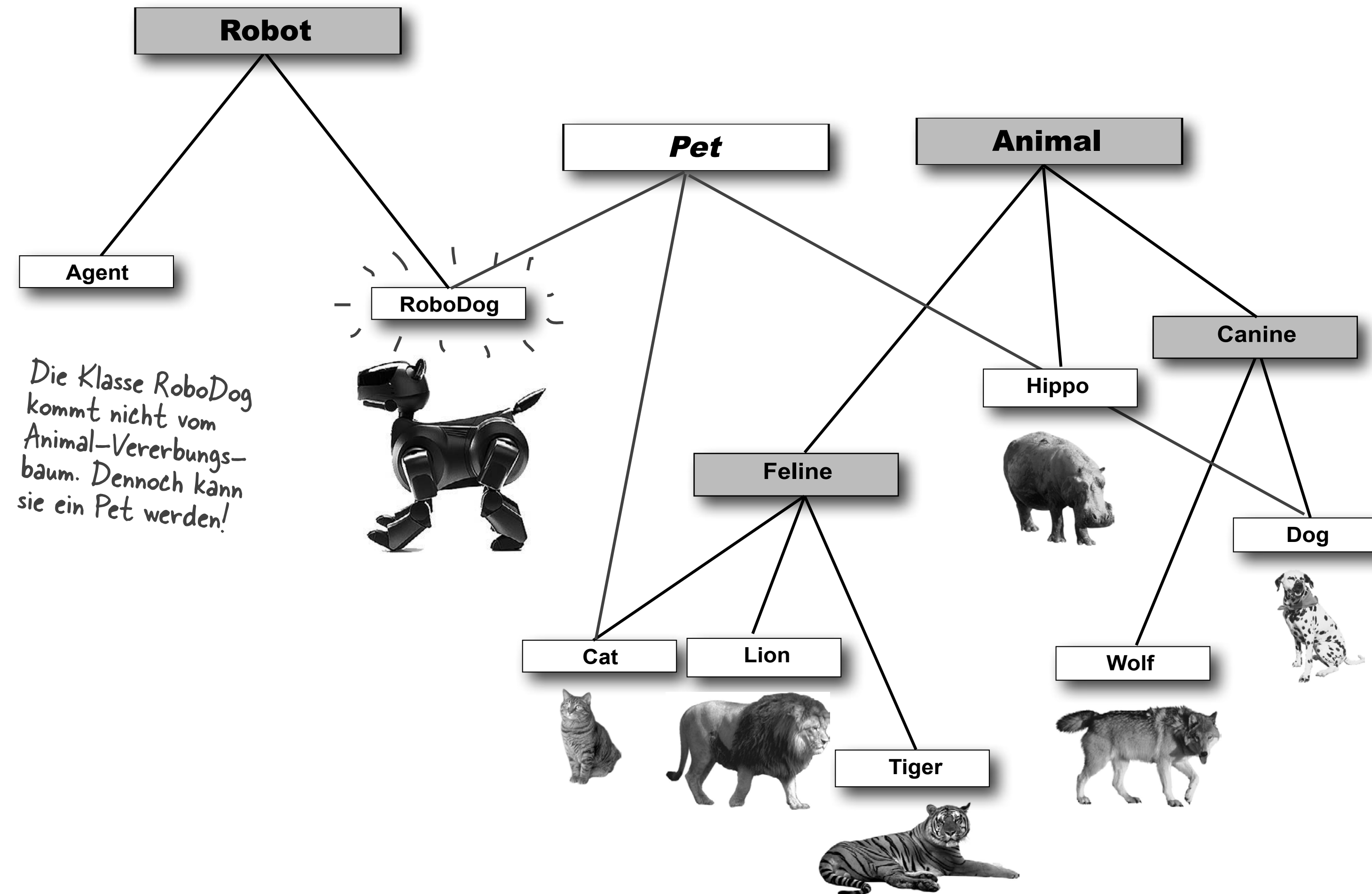
```
public class Dog extends Canine implements Pet {  
    public void beFriendly() { ... }  
    public void play() { ... }  
  
    public void roam() { ... }  
    public void eat() { ... }  
}
```

Durch die
Implementierung
der Methoden wird
der Vertrag „IST EIN
Pet“ eingehalten.

Unterschiedliche Vererbungsbäume

Interfaces

Klassen von *unterschiedlichen* Vererbungsbäumen können das *gleiche* Interface implementieren.



„Und eine Klasse kann sogar mehrere Interfaces implementieren.“

```
public class Batman extends Superhero  
    implements HasSidekick,  
    HasDeductiveSkill, IsDC { ... }
```

Gerit – aufgeweckte Student:in im 1. Semester



„Wann verwende ich normale Klassen,
Unterklassen, abstrakte Klassen oder
Interfaces?“

Noel – neugierige Student:in im 1. Semester



Klasse, Unterklasse, Abstrakte Klasse oder Interface?

Interfaces

- Erstellen Sie eine **Klasse**, die nichts erweitert (außer Object), wenn Ihre neue Klasse keinen IST-EIN-Test für irgendeinen anderen Typ besteht.
- Erstellen Sie eine **Unterklasse** (erweitern Sie eine Klasse) nur dann, wenn Sie eine spezifischere Version einer Klasse benötigen und Verhalten überschreiben oder hinzufügen müssen.
- Verwenden Sie eine **abstrakte Klasse**, wenn Sie eine Vorlage für eine Gruppe von Unterklassen definieren wollen und es zumindest etwas Implementierungscode gibt, den alle Unterklassen gemeinsam nutzen sollen. Machen Sie die Klasse abstrakt, wenn Sie sicherstellen wollen, dass niemand Objekte dieses Typs erzeugen kann.
- Verwenden Sie ein **Interface**, wenn Sie eine Rolle definieren möchten, die Klassen unabhängig von ihrer Position im Vererbungsbaum spielen können.

! Quiz

Interfaces

-  Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
-  Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
-  Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Ein Interface wird mit den Schlüsselwörtern
`interface` `class` anstatt nur `class`
erstellt.

... mit dem Schlüsselwort `interface` ...

Um ein Interface zu implementieren, verwenden
Sie das Schlüsselwort `implements`.
Beispiel: `Dog implements Pet`

Richtig!

Eine Klasse kann wegen des Deadly Diamond of Death nur ein Interface implementieren.

Eine Klasse kann mehr als ein Interface implementieren.

Wenn eine konkrete Klasse ein Interface implementiert, muss sie alle Methoden des Interfaces implementieren – mit Ausnahme der Methoden, die, wie wir später sehen werden, als `default` und `static` gekennzeichnet sind.

Richtig!

 Welcher **eine** Gedanke zu
„ Interfaces“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren

Pair Programming

🎓 interface

🤝 Pair Programming

- 🎯 Erstellen Sie das Interface `Superheroic` und zwei Klassen, die das Interface implementieren.
- ✅ Schreiben Sie ein Interface `Superheroic` mit der Methode `void useSuperPower()`.
- ✅ Realisieren Sie zwei Klassen, die jeweils eine Superheld:in Ihrer Wahl repräsentieren und dabei das Interface `Superheroic` implementieren.



interface

Pair Programming

 **Erstellen Sie das Interface `Superheroic` und zwei Klassen, die das Interface implementieren.**

 Schreiben Sie ein Interface `Superheroic` mit der Methode `void useSuperPower()`.

 Realisieren Sie zwei Klassen, die jeweils eine Superheld:in Ihrer Wahl repräsentieren und dabei das Interface `Superheroic` implementieren.

 Die konkreten Superheld:innen sollen ihre jeweiligen Superkräfte einsetzen.

 Es genügt, wenn die Implementierung der Methode im Wesentlichen aus einer `println`-Anweisung besteht.

 Schreiben Sie eine `main()`-Methode, die die Methode `useSuperPower()` über Referenzvariablen vom Typ `Superheroic` aufruft.

 Auf der Konsole werden die charakteristischen Superkräfte angezeigt.

```
1 import java.util.*;
2
3 public class SuperheroesDemo {
4     public static void main(String[] args) {
5
6         List<Superheroic> heroes = List.of(new Superman(), new TheFlash()); // unmodifiable List
7
8         for (Superheroic hero : heroes) {
9             hero.useSuperPower();
10        }
11
12    }
13 }
14
```

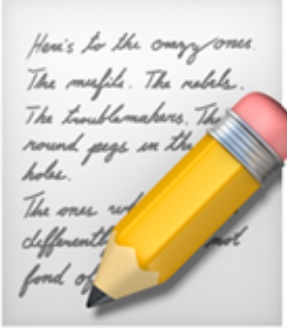
STDIN

Input for the program (Optional)

Output:

As Superman, I use my super strength and x-ray vision!

As The Flash, I use my incredible speed and superhuman agility!

 Welcher **eine** Gedanke zu
„ interface“ war für Sie
am wichtigsten?  Schreiben
Sie ihn auf!

 Durchatmen und  Sortieren

Superklassenversion

„Wenn ich eine Methode überschreibe, wie kann ich dem Verhalten der Methode in der Superklasse etwas hinzufügen, anstatt das dortige Verhalten komplett mit meinem Code zu ersetzen?“

Noel – neugierige Student:in im 1. Semester




```
1 public class SuperDemo {
2     public static void main(String[] args) {
3
4         Car myCar = new Car();
5         myCar.startEngine();
6
7     }
8 }
9
```

STDIN

Input for the program (Optional)

Output:

Starting a car engine...
Performing car-specific start checks.
Performing basic engine start operations.
Car engine started.

! ? Quiz

UP! Superklassenversion

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Um die Superklassenversion einer Methode von einer Unterklasse aus aufzurufen, die diese Methode überschrieben hat, verwenden Sie das Schlüsselwort `super`.
Beispiel: `super.showCourage()` ;

Richtig!

Das Schlüsselwort `super` ist eigentlich ein Verweis auf den Superklassenteil des Objekts. Wenn der Code der Unterklasse `super` verwendet, wird die Superklassenversion des Codes aufgerufen.

Richtig!

 Welcher **eine** Gedanke zu
„ Superklassenversion“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Interfaces und abstrakte Klassen

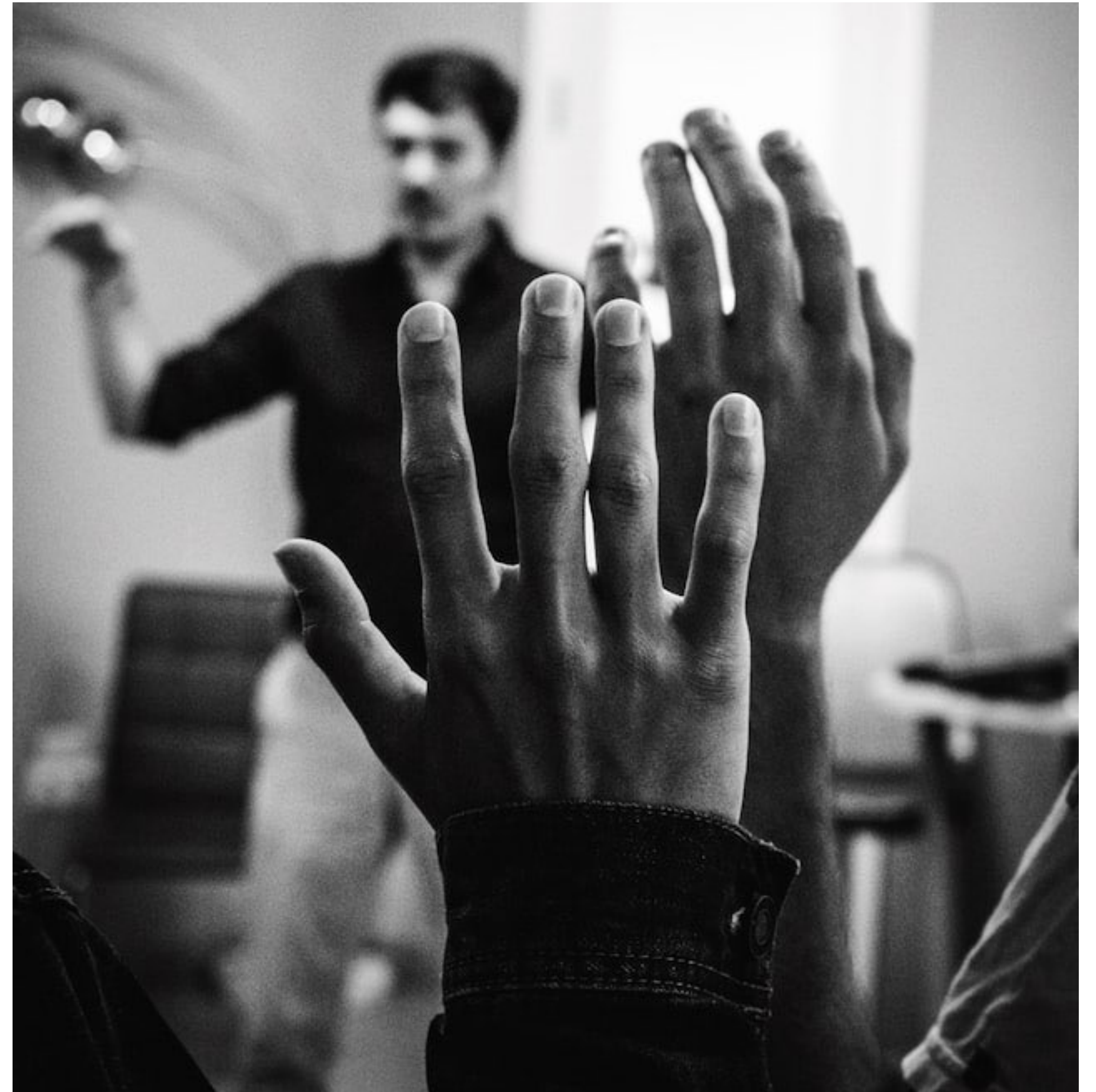
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Interfaces und abstrakte Klassen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Interfaces und abstrakte Klassen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus



„Ich wünsche Ihnen erholsame Feiertage
und einen schwungvollen Start ins neue
Jahr!“



Martin Burger